

Intégration blockchain

SEGHO-WALE — Centre Médico-Social WALÉ

Garantir l'intégrité, la traçabilité et la sécurité des données médicales, et une interopérabilité fiable entre les acteurs de la prise en charge du patient (centre de santé, assureurs, régulateur, laboratoire).

Document technique — Hyperledger Fabric (chaincode Go) + intégration Django

Intégration blockchain — SEGHO-WALE

Ce document décrit l'intégration blockchain ajoutée à SEGHO-WALE : ce qui a été construit, pourquoi, ce qu'il reste à faire, et comment tester chaque partie — y compris ce qui peut être vérifié dès maintenant, sans réseau Hyperledger Fabric actif.

1. Objectif et principe général

Le thème du projet est d'ajouter une couche blockchain à SEGHO-WALE pour garantir l'**intégrité**, la **traçabilité** et la **sécurité** des données médicales, et une **interopérabilité fiable** entre les différents acteurs de la prise en charge du patient : le centre de santé, les assureurs, le régulateur (Ministère de la Santé), et — cas le plus concret déjà en production dans SEGHO-WALE — le laboratoire externe (échange HPRIM).

Principe central : le hash sur la chaîne, les données dans Django. Aucune donnée médicale en clair ne transite sur le registre partagé. Seuls sont ancrés : une **empreinte SHA-256** de l'enregistrement, son type, un identifiant métier (`PAT...` , `CONS...` , `FAC...`) et des métadonnées non sensibles (centre, horodatage). Les dossiers complets restent dans la base Django de chaque centre. La blockchain sert à **prouver, entre organisations qui ne se font pas mutuellement confiance, qu'un enregistrement n'a pas été altéré depuis son ancrage** — pas à le stocker.

Technologie retenue : **Hyperledger Fabric**, un réseau blockchain permissionné à **3 organisations** :

Organisation	MSP ID	Rôle
Centre de santé (CMS WALÉ)	CentreSanteMSP	soumet l'essentiel des événements (dossiers patients, consultations, ordonnances, factures, échanges laboratoire)
Assurance	AssuranceMSP	vérifie les faits liés à la facturation/remboursement sans voir le dossier médical complet
Régulateur (Ministère de la Santé)	RegulateurMSP	audit, déclarations épidémiologiques obligatoires, registre de vaccination

Les 3 organisations partagent un canal unique, `canal-sante` : comme seules des empreintes et métadonnées y circulent, un canal unique suffit (pas de collection de données privées nécessaire au stade actuel).

2. Ce qui a été ajouté

2.1 Le chaincode Go — `blockchain/chaincode/dossier-medical/`

Le « smart contract » du réseau, écrit en Go avec le framework `fabric-contract-api-go`. Trois contrats, chacun couvrant une des exigences du thème :

- **IntegriteContract** (intégrité + traçabilité) — ancre une empreinte (`EnregistrerEvenement`) de façon **immuable** : aucune fonction de mise à jour ou de suppression n'est exposée, un identifiant déjà utilisé est refusé. Permet de consulter l'historique complet d'une entité (`HistoriqueEntite`), de lister les événements d'un centre (`EvenementsParCentre` , requête riche CouchDB), et de **vérifier l'intégrité** d'un enregistrement (`VerifierIntegrite` compare le hash ancré à un hash recalculé depuis la base Django actuelle).
- **ConsentementContract** (sécurité / droits du patient) — octroi et révocation d'autorisations d'accès qu'un patient (ou le centre en son nom) accorde à une autre organisation, sur un périmètre et une durée donnés.
- **TransfertContract** (interopérabilité inter-centres) — notarie un transfert de patient entre deux centres : le centre destinataire confirme la réception avec l'empreinte du dossier reçu, comparée à celle notariée par le centre d'origine, prouvant l'absence d'altération pendant le transfert.

Chaque contrat est accompagné de **tests unitaires** (`*_test.go`) utilisant des doubles de test écrits à la main (`chaincode/mocks/`) — ils s'exécutent avec `go test` , **sans avoir besoin de Docker ni d'un réseau Fabric réel**.

2.2 Le réseau Fabric — `blockchain/network/`

Configuration complète pour faire tourner un réseau à 3 organisations en local avec Docker :

- `configtx.yaml` — définit le consortium et le canal `canal-sante` .
- `crypto-config.yaml` — matériel cryptographique des 3 organisations + de l'orderer (généré par l'outil `cryptogen`).
- `docker-compose.yaml` — un orderer, un peer par organisation, une base d'état CouchDB par peer (nécessaire aux requêtes riches), un conteneur CLI.
- `scripts/generate.sh` et `scripts/network.sh` — scripts d'orchestration (génération du matériel crypto, démarrage du réseau, création du canal, déploiement du chaincode).

2.3 La passerelle REST — `blockchain/gateway/`

Un petit service Go qui expose les 3 smart contracts en HTTP, pour que Django puisse les appeler sans dépendre d'un SDK Fabric en Python (peu maintenu). Une instance de la passerelle sert une organisation ; celle utilisée par Django est l'instance côté `CentreSanteMSP` .

Endpoints principaux : `POST /evenements` , `GET /evenements/{id}` , `POST /evenements/{id}/verifier` , `GET /entites/{type}/{id}/historique` , ainsi que les équivalents pour `/consentements` et `/transferts` .

2.4 L'intégration Django — `blockchain_bridge/`

Nouvelle application Django, câblée à l'existant **uniquement via des signaux** (aucune vue ni template modifié dans les autres apps) :

- `AncreBlockchain` (modèle) — trace chaque événement ancré : type d'entité, code métier, empreinte, identifiant de transaction, statut (`en_attente` / `confirmé` / `erreur`).
- `hashing.py` — calcule une empreinte canonique par type d'entité (sérialisation JSON triée puis SHA-256), et sait **recalculer** cette empreinte à partir de l'état courant de la base pour la vérification d'intégrité.
- `services.py` — `ancrer_evenement(...)` (appelle la passerelle, idempotent, jamais bloquant) et `verifier_integrite(...)` (recalcule et compare).
- `signals.py` — déclenche l'ancrage sur des **transitions significatives**, pas à chaque sauvegarde brouillon.
- `admin.py` — consultation en lecture seule des ancrages, avec une action « **Vérifier l'intégrité** ».
- Réglages `BLOCKCHAIN_ENABLED` (désactivé par défaut) et `BLOCKCHAIN_GATEWAY_URL` dans `medisoft/settings.py` .

Important : toute panne réseau vers la blockchain est journalisée, jamais levée. Une indisponibilité du réseau Fabric ne bloque jamais un soin, une facturation ou un import HPRIM — c'est vérifié par les tests automatisés.

Entités ancrées et déclencheurs

Entité	Type (<code>type_entite</code>)	Déclencheur
<code>patients.Patient</code>	<code>patient</code>	à la création
<code>consultations.Consultation</code>	<code>consultation</code>	passage du statut à <code>termine</code> (inclut diagnostics, constantes, ordonnances de la consultation)
<code>consultations.Ordonnance</code>	<code>ordonnance</code>	émission hors consultation
<code>facturation.Facture</code>	<code>facture</code>	passage du statut à <code>emise</code>
<code>facturation.Paiement</code>	<code>paiement</code>	à la création
<code>laboratoire.EchangeHPRIM</code>	<code>echange_hprim</code>	envoi transmis (ORM) ou réception traitée (ORU/ERR) — voir § 2.5
<code>laboratoire.AnalyseLaboratoire</code>	<code>analyse_laboratoire</code>	résultat validé, une fois ses lignes de résultat attachées — voir § 2.5

2.5 Point central : l'échange HPRIM avec le laboratoire

C'est le cas d'interopérabilité inter-systèmes le plus concret du projet : SEGHO-WALE échange par FTP, au format **HPRIM Santé v2.4**, des demandes d'examens (`ORM`) et des résultats (`ORU` , ou une erreur `ERR`) avec le système du laboratoire externe (SYSLAM) — deux logiciels différents, deux acteurs différents, aucune confiance mutuelle a priori. C'est exactement le scénario que `IntegriteContract` sert à sécuriser.

- **EchangeHPRIM** — chaque message *réellement* échangé est ancré : l'envoi d'une demande une fois transmise par FTP (`sens='envoi'` , `statut='transmis'`), et la réception d'un résultat ou d'une erreur une fois intégrée (`sens='reception'` , `statut='traite'`). Les tentatives internes en échec (FTP indisponible, configuration absente) ne sont **pas** ancrées : rien n'a alors franchi la frontière inter-systèmes. L'empreinte porte sur le **contenu brut du message HPRIM** — exactement les octets transmis au/reçus du laboratoire, avant toute interprétation côté SEGHO.
- **AnalyseLaboratoire** — une fois un résultat validé (`statut='valide'`), le résultat structuré côté SEGHO (paramètres, valeurs, unités, interprétation) est ancré à son tour. Cet ancrage est déclenché **explicitement** depuis `laboratoire/hprim/integration.py` , juste après que les résultats ont été attachés à l'analyse — pas via un signal générique : au moment où le statut passe à `valide` pendant un import HPRIM, les résultats ne sont pas encore créés (ils le sont juste après, dans la même fonction). Un ancrage sur un simple signal aurait donc figé un jeu de résultats **incomplet**, de façon permanente (le registre étant immuable). Ce point est couvert par un test dédié (§ 4.2).

Résultat : une chaîne de traçabilité vérifiable à chaque étape — *demande envoyée* → *résultat brut reçu du labo* → *résultat validé cliniquement par SEGHO*.

2.6 Fichiers existants modifiés

Fichier	Modification
<code>medisoft/settings.py</code>	ajout de <code>'blockchain_bridge'</code> à <code>INSTALLED_APPS</code> , des réglages <code>BLOCKCHAIN_ENABLED</code> / <code>BLOCKCHAIN_GATEWAY_URL</code>
<code>requirements.txt</code>	ajout de <code>requests</code> (appel HTTP vers la passerelle)
<code>laboratoire/hprim/integration.py</code>	un appel explicite à l'ancrage de <code>AnalyseLaboratoire</code> , juste après l'attachement des résultats d'un ORU (voir § 2.5)

Aucun autre fichier existant n'a été modifié : les apps `patients` , `consultations` , `facturation` ne sont touchées que par les **signaux** de `blockchain_bridge` , pas par une modification de leur code.

3. Ce qui n'a pas encore été fait (limites connues)

- **Le réseau Fabric n'a jamais été démarré ni compilé dans cet environnement de développement** : ni Docker, ni un accès internet vers le registre de modules Go (`proxy.golang.org`) n'y étaient disponibles. Le chaincode et la passerelle ont été écrits et relus avec soin, mais `go build` / `go test` sur ce code précis n'a **pas** pu être exécuté ici — c'est la toute première chose à faire une fois Docker/internet disponibles (voir § 5 et § 6).
 - **ConsentementContract** et **TransfertContract** **sont pleinement implémentés et exposés par la passerelle**, mais ne sont reliés à aucune vue Django : les workflows métier correspondants (recueil de consentement patient, transfert inter-centres) n'existent pas encore dans l'application. Les brancher est une extension naturelle du travail déjà fait, pas une réinvention.
 - **BLOCKCHAIN_ENABLED** vaut `False` **par défaut** : tant qu'il n'est pas activé, aucun appel réseau n'est effectué (voir § 6.1 pour l'activer).
 - Un problème préexistant, **sans rapport avec la blockchain**, a été repéré pendant les tests : 10 tests échouent dans plusieurs apps (`stock` compris) sur des contrôles de permissions (`403` au lieu de `302` attendu). Confirmé reproductible sans aucune des modifications de cette intégration — signalé pour information, non corrigé ici (hors périmètre demandé).
-

4. Comment tester ce qui a été ajouté

4.1 Sans Docker ni réseau Fabric (faisable dès maintenant)

a) **Le chaincode Go**, avec ses tests unitaires (mocks, aucun réseau requis) :

```
cd blockchain/chaincode/dossier-medical
go mod tidy      # première fois : télécharge fabric-contract-api-go, etc.
go build ./...
go vet ./...
go test ./...
```

b) **La passerelle Go**, compilation seule (pas de peer à contacter) :

```
cd blockchain/gateway
go mod tidy
go build ./...
```

c) **L'application Django**, migrations et tests automatisés :

```
python manage.py makemigrations blockchain_bridge # doit afficher "No changes detected"
python manage.py migrate
python manage.py check
python manage.py test blockchain_bridge laboratoire patients consultations facturation
```

Résultat attendu : tous les tests passent. Les tests de `blockchain_bridge` utilisent des doubles (`unittest.mock`) pour l'appel réseau vers la passerelle — ils vérifient le **déclenchement** correct des ancrages (quelles transitions l'activent, lesquelles ne l'activent pas), pas la connexion Fabric réelle.

d) Vérification manuelle dans l'admin, sans réseau actif — avec `BLOCKCHAIN_ENABLED=False` (valeur par défaut), créez un patient, terminez une consultation, émettez une facture : le comportement de l'application ne doit **strictement rien changer** (aucun appel réseau, aucune erreur). C'est la garantie que l'intégration est totalement transparente tant qu'elle n'est pas activée.

4.2 Test ciblé sur l'échange HPRIM (le point le plus important)

Le fichier `blockchain_bridge/tests.py` contient des tests dédiés à ce scénario, à lire pour comprendre le comportement attendu :

- `AncrageEchangeHPRIMTests` : vérifie qu'un envoi `transmis` déclenche l'ancrage, qu'un envoi en `erreur` ne le déclenche pas, qu'une réception `traite` le déclenche, qu'une réception simplement `recu` (pas encore traitée) ne le déclenche pas.
- `AncrageAnalyseLaboratoireTests` : vérifie que l'empreinte d'une analyse change bien selon que ses résultats sont attachés ou non (`test_empreinte_inclut_les_resultats_attaches`), et que l'ancrage explicite utilise bien l'état complet.

Les lancer isolément :

```
python manage.py test blockchain_bridge.tests.AncrageEchangeHPRIMTests
python manage.py test blockchain_bridge.tests.AncrageAnalyseLaboratoireTests
```

4.3 Avec le réseau Fabric réellement démarré (une fois Docker disponible)

1. Générer le matériel cryptographique et les artefacts du canal :

```
bash cd blockchain/network ./scripts/generate.sh
```

1. Démarrer le réseau, créer le canal, déployer le chaincode :

```
bash ./scripts/network.sh up ./scripts/network.sh createChannel ./scripts/network.sh
deployCC
```

1. Démarrer la passerelle (côté `CentreSanteMSP`) :

```
bash cd blockchain/gateway cp config.example.yaml config.yaml # adapter les chemins si
besoin go run . -config config.yaml
```

Vérifier qu'elle répond : `curl http://localhost:8090/sante` doit renvoyer `{"statut":"ok"}`.

1. Activer l'intégration côté Django (`.env` ou variables d'environnement) :

```
BLOCKCHAIN_ENABLED=True BLOCKCHAIN_GATEWAY_URL=http://localhost:8090
```

1. **Test de bout en bout** : dans l'admin Django, créer un patient. Aller dans **Blockchain** →

Ancrages blockchain : une ligne doit apparaître avec `statut=Confirmé` et un `tx_id` renseigné.

Ouvrir cette ligne, sélectionner l'action « **Vérifier l'intégrité** » : le message doit confirmer l'intégrité.

2. **Test de détection d'altération** : modifier directement en base (ou via l'admin, si le champ est éditable) une donnée déjà ancrée d'un patient (ex. son nom), puis relancer l'action « **Vérifier l'intégrité** » sur l'ancrage correspondant : le message doit signaler une **altération détectée** — c'est la preuve que le mécanisme fonctionne.
3. **Test du scénario HPRIM complet** : avec une configuration HPRIM active pointant vers un serveur FTP de test (ou le vrai SYSLAM), créer une `DemandeExamen` et la faire passer au statut *Demandé* ; vérifier l'apparition d'un ancrage `echange_hprim` une fois l'envoi transmis ; à la réception d'un résultat (`relever_resultats`), vérifier l'apparition d'un second ancrage `echange_hprim` (réception) puis d'un ancrage `analyse_laboratoire` une fois le résultat validé.

5. Étapes suivantes à faire

1. **Installer les prérequis** : Docker Desktop, Git Bash ou WSL2, les binaires Fabric (`cryptogen` , `configtxgen` , `peer`) — voir l'en-tête de `blockchain/network/scripts/generate.sh` pour la commande d'installation officielle.
2. **Compiler et tester le chaincode et la passerelle** (§ 4.1.a et 4.1.b) — première vérification réelle de ce code, non exécutable dans l'environnement où il a été écrit.
3. **Démarrer le réseau et dérouler le test de bout en bout** (§ 4.3).
4. **Activer** `BLOCKCHAIN_ENABLED` en environnement de démonstration/test, et observer les ancres se créer au fil de l'usage normal de l'application (création de patients, consultations, factures, et surtout échanges HPRIM).
5. **Rédiger la partie du mémoire consacrée aux résultats** en s'appuyant sur des captures d'écran de l'admin (`Ancrages blockchain`), et si possible du registre Fabric lui-même (ex. `peer chaincode query` sur `HistoriqueEntite`).
6. **Extensions possibles**, une fois la base validée : - brancher `ConsentementContract` à un écran de gestion du consentement patient ; - brancher `TransfertContract` le jour où un workflow de transfert inter-centres est ajouté à l'application ; - passer de `cryptogen` à une Fabric CA par organisation, pour un provisionnement d'identités plus réaliste ; - passer l'ordre à 3 nœuds (tolérance aux pannes) ; - étendre l'ancrage aux déclarations épidémiologiques obligatoires et au registre de vaccination (déjà modélisés dans `rapports`), qui intéressent directement `OrgRegulateur` .
7. **(Optionnel, hors périmètre blockchain)** : investiguer les 10 échecs de tests préexistants liés aux permissions (403 au lieu de 302), signalés en § 3.

6. Annexe technique

6.1 Arborescence des fichiers ajoutés

```
blockchain/
  README.md
  chaincode/dossier-medical/
    go.mod, main.go
    chaincode/
      types.go
      integrite_contract.go      + integrite_contract_test.go
      consentement_contract.go  + consentement_contract_test.go
      transfert_contract.go      + transfert_contract_test.go
    mocks/mocks.go
  network/
    configtx.yaml, crypto-config.yaml, docker-compose.yaml
    scripts/generate.sh, scripts/network.sh
  gateway/
    go.mod, main.go, fabric_client.go, handlers.go, config.example.yaml

blockchain_bridge/
  apps.py, models.py, hashing.py, services.py, signals.py, admin.py, tests.py
  migrations/0001_initial.py, 0002_alter_ancrageblockchain_type_entite.py
```

6.2 Glossaire

Terme	Signification
Chaincode	Le « smart contract » Hyperledger Fabric : le code métier qui s'exécute sur chaque peer et définit les règles d'écriture/lecture du registre.
MSP (Membership Service Provider)	L'identité cryptographique d'une organisation sur le réseau Fabric.
Peer	Un nœud du réseau Fabric qui exécute le chaincode et maintient une copie du registre.
Ledger (registre)	La base de données répliquée et immuable tenue par le réseau.
Ancrage	L'action d'inscrire une empreinte sur le registre (<code>EnregistrerEvenement</code>).
HPRIM	Norme française d'échange de données de santé (ASTM E1238), utilisée ici pour les échanges avec le laboratoire (contextes <code>ORM</code> = demande, <code>ORU</code> = résultat, <code>ERR</code> = erreur).
SYSLAM	Le logiciel du laboratoire externe avec lequel SEGHO-WALE échange en HPRIM.